

National University of Computer and Emerging Sciences



AL-2002: Artificial Intelligence Lab Lab Manual 13

Department of Computer Science
FAST-NU, Lahore, Pakistan

Table of Contents

Objectives	3
1 Introduction to Hugging Face Datasets	3
1.1 What is the Hugging Face Datasets Library?	4
1.2 Loading a Dataset	4
1.3 Dataset Preprocessing	6
2 Tokenization and Dataset Preparation	7
3 Fine-Tuning a Pretrained Model	8
3.1 Trainer API	9
4 Model Evaluation	10

Objectives

After performing this lab, students shall be able to:

- ✓ Understand the Hugging Face Datasets library
- ✓ Load preprocess and train datasets
- ✓ Tokenize the dataset and prepare for training
- ✓ Fine tuning a pre trained model
- ✓ Model evaluation

1 Introduction to Hugging Face Datasets

Modern machine learning models require large amounts of data for training and evaluation. The Hugging Face Datasets library provides an efficient and convenient way to access and manage datasets for machine learning tasks.

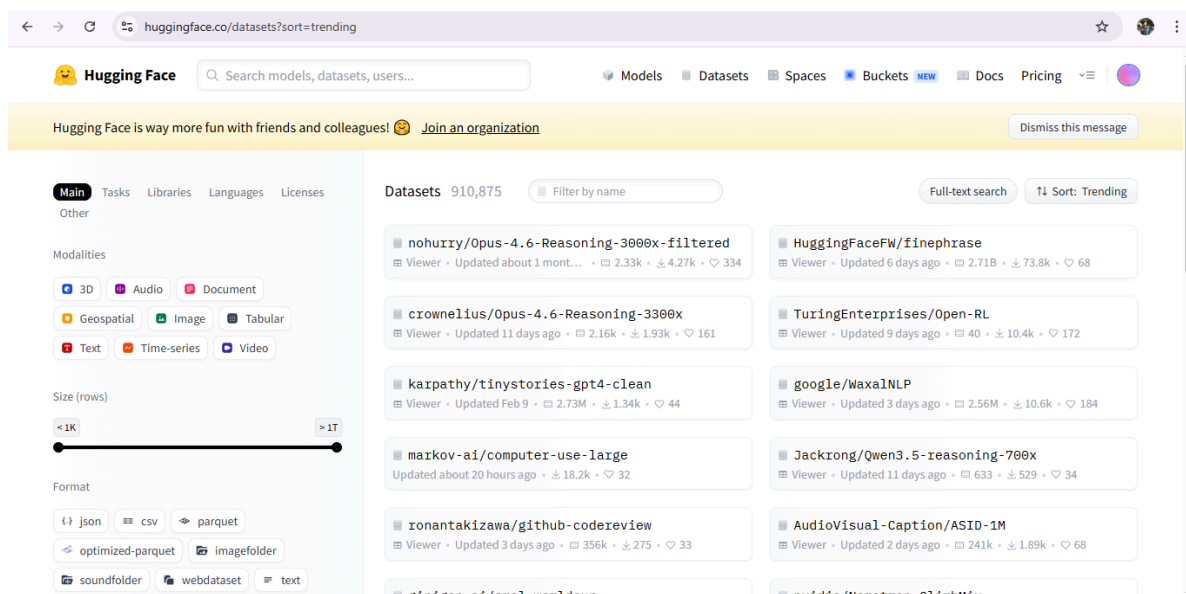
The library provides tools to:

- Download and load datasets easily
- Process large datasets efficiently
- Handle text, images, audio, and structured data
- Integrate directly with Hugging Face Transformers

The datasets library is designed to work seamlessly with PyTorch, TensorFlow, and JAX frameworks.

To install the library:

```
pip install datasets
```



1.1 What is the Hugging Face Datasets Library?

The Hugging Face datasets library is an open-source tool that provides access to thousands of ready-to-use datasets for machine learning.

Key features include:

- Access to thousands of datasets
- Efficient data loading using Apache Arrow
- Built-in dataset processing tools
- Integration with Transformers training pipelines
- Memory-efficient handling of large datasets

Datasets can be used for tasks such as:

- Text classification
- Machine translation
- Question answering
- Named entity recognition
- Sentiment analysis

1.2 Loading a Dataset

Datasets can be loaded using the `load_dataset()` function.

```
from datasets import load_dataset
dataset = load_dataset("imdb")
```

The IMDB dataset is commonly used for sentiment analysis.

The dataset contains two main splits:

- Training set
- Test set

To view the dataset structure:

```
print(dataset)
```

```
DatasetDict({
  train: Dataset({
```

```

        features: ['text', 'label'],

        num_rows: 25000

    })

    test: Dataset({

        features: ['text', 'label'],

        num_rows: 25000

    })

    unsupervised: Dataset({

        features: ['text', 'label'],

        num_rows: 50000

    })

})

```

Once the dataset is loaded, we can inspect its contents.

View a sample entry:

```
print(dataset["train"][0])
```

{'text': 'I rented I AM CURIOUS-YELLOW from my video store because of all the controversy that surrounded it when it was first released in 1967. I also heard that at first it was seized by U.S. customs if it ever tried to enter this country, therefore being a fan of films considered "controversial" I really had to see this for myself.

The plot is centered around a young Swedish drama student named Lena who wants to learn everything she can about life. In particular she wants to focus her attentions to making some sort of documentary on what the average Swede thought about certain political issues such as the Vietnam War and race issues in the United States. In between asking politicians and ordinary denizens of Stockholm about their opinions on politics, she has sex with her drama teacher, classmates, and married men.

What kills me about I AM CURIOUS-YELLOW is that 40 years ago, this was considered pornographic. Really, the sex and nudity scenes are few and far between, even then it\'s not shot like some cheaply made porno. While my countrymen mind find it shocking, in reality sex and nudity are a major staple in Swedish cinema. Even Ingmar Bergman, arguably their answer to good old boy John Ford, had sex scenes in his films.

I do commend the filmmakers for the fact that any sex shown in the film is shown for artistic purposes rather than just to shock people and make money to be shown in pornographic theaters in America. I AM

CURIOUS-YELLOW is a good film for anyone wanting to study the meat and potatoes (no pun intended) of Swedish cinema. But really, this film doesn't have much of a plot.',

"label": 0}

```
print(dataset["train"].features)
```

```
{'text': Value('string'), 'label': ClassLabel(names=['neg', 'pos'])}
```

1.3 Dataset Preprocessing

Raw datasets usually require preprocessing before they can be used for training.

Common preprocessing steps include:

- Cleaning text
- Tokenization
- Removing unnecessary columns
- Converting labels into numerical format

The datasets library provides the **map()** function for applying transformations to the dataset.

```
def preprocess(example):
    return {"text_length": len(example["text"])}

dataset = dataset.map(preprocess)
```

This function adds a new column called `text_length`.

```
DatasetDict({
    train: Dataset({
        features: ['text', 'label', 'text_length'],
        num_rows: 25000
    })
    test: Dataset({
        features: ['text', 'label', 'text_length'],
        num_rows: 25000
    })
})
```

```

unsupervised: Dataset({
    features: ['text', 'label', 'text_length'],
    num_rows: 50000
})
})

```

2 Tokenization and Dataset Preparation

Transformer models cannot process raw text directly. The text must first be converted into tokens using a tokenizer. We use the tokenizer corresponding to the pretrained model.

```

from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
def tokenize_function(example):
    return tokenizer(example["text"], padding="max_length",
truncation=True)
tokenized_datasets = dataset.map(tokenize_function, batched=True)

```

- padding="max_length" ensures all sequences have the same length
- truncation=True cuts text exceeding the maximum length
- batched=True processes multiple examples at once for efficiency

After tokenization, the dataset will contain additional fields such as:

- input_ids
- token_type_ids
- attention_mask

These are required inputs for Transformer models.

```
print(tokenized_datasets)
```

```

DatasetDict({
    train: Dataset({
        features: ['text', 'label', 'text_length', 'input_ids',
'token_type_ids', 'attention_mask'],
        num_rows: 25000
    })
})

```

```

test: Dataset({
    features: ['text', 'label', 'text_length', 'input_ids',
'token_type_ids', 'attention_mask'],
    num_rows: 25000
})

unsupervised: Dataset({
    features: ['text', 'label', 'text_length', 'input_ids',
'token_type_ids', 'attention_mask'],
    num_rows: 50000
})
})

```

After tokenization, we prepare the dataset for training we need to remove unnecessary columns and rename the label column if necessary:

```

tokenized_datasets = tokenized_datasets.remove_columns(["text"])
tokenized_datasets = tokenized_datasets.rename_column("label", "labels")

```

Set the dataset format to PyTorch tensors:

```

tokenized_datasets.set_format("torch")

```

Now the dataset is ready for training with Transformers.

3 Fine-Tuning a Pretrained Model

Fine-tuning means taking a pretrained model and training it further on a specific dataset to adapt it to a particular task. We will fine-tune a model for sentiment classification.

```

from transformers import AutoModelForSequenceClassification
model =
AutoModelForSequenceClassification.from_pretrained("distilbert-base-unca
sed", num_labels=2)

```

num_labels=2 indicates binary classification (positive or negative).

3.1 Trainer API

The Hugging Face Trainer API simplifies the training process by handling:

- Training loops
- Evaluation
- Logging
- Saving models
- Hardware acceleration

First define training arguments.

```
from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir="results",
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=3,
)
```

- `output_dir`: where model checkpoints are saved
- `learning_rate`: step size used by the optimizer
- `batch_size`: number of samples processed per batch
- `num_train_epochs`: number of times the dataset is passed through the model

These are just a few parameters. Explore other here:

https://huggingface.co/docs/transformers/v5.3.0/en/main_classes/trainer#transformers.TrainingArguments

Next we create a Trainer object.

```
from transformers import Trainer

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["test"]
)
```

The Trainer connects the model, dataset and training configuration.

To start training:

```
trainer.train()
```

Example output from training on a subset of the dataset:

```
TrainOutput(global_step=75, training_loss=0.045988521575927734,  
metrics={'train_runtime': 43.8771, 'train_samples_per_second': 13.675,  
'train_steps_per_second': 1.709, 'total_flos': 79480439193600.0,  
'train_loss': 0.045988521575927734, 'epoch': 3.0})
```

During training, the Trainer will:

- Load batches of data
- Compute predictions
- Calculate loss
- Update model weights

After training, the fine-tuned model will perform better on the specific task.

4 Model Evaluation

Evaluating the model helps measure how well it performs on unseen data.

Common classification metrics include:

- Accuracy
- Precision
- Recall
- F1 Score

We can implement a metric function using the evaluate library.

```
pip install evaluate
```

Then define the metric computation.

```
import evaluate  
import numpy as np  
  
metric = evaluate.load("accuracy")
```

```
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    return metric.compute(predictions=predictions, references=labels)
```

Add this function to the Trainer.

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["test"],
    compute_metrics=compute_metrics
)
```

Now evaluate the model:

```
trainer.evaluate()
```

```
{'eval_loss': 5.006828784942627,
 'eval_model_preparation_time': 0.0025,
 'eval_accuracy': 0.52,
 'eval_runtime': 2.9189,
 'eval_samples_per_second': 68.519,
 'eval_steps_per_second': 8.565,
 'epoch': 3.0}
```

This indicates the model correctly classified 52% of the test examples (The accuracy is low because the model was trained only on a small subset of the dataset for demonstration purposes)

In case you have a separate validation dataset, set it as `eval_dataset` in `Trainer()` and then use `trainer.evaluate(tokenized_dataset["test"])` to get performance of the test dataset.